

Assessment I

Vorname: _____

Punkte: ____ / 90, Note: ____

Name: _____

Frei lassen für Korrektur.

Klasse: 4ibb1

Hilfsmittel:

- Notizen, ein A4-Blatt, Font `Courier New`, ≥ 10 pt.
- Löse die Aufgaben jeweils direkt auf den Prüfungsblättern.
- Zusatzblätter, falls nötig, mit Namen und Frage-Nr. auf jedem Blatt.

Nicht erlaubt:

- Unterlagen (Slides, Bücher, ...).
- Computer (Laptop, Smartphone, ...).
- Kommunikation (mit Personen, KI, ...).

Bewertung:

- Multiple Response: ☐ *Ja* oder ☐ *Nein* ankreuzen, +1/-1 Punkt pro richtige/falsche Antwort, beide nicht ankreuzen ergibt +0 Punkte; Total pro Frage gibt es nie weniger als 0 Punkte.
- Offene Fragen: Bewertet wird Korrektheit, Vollständigkeit und Kürze der Antwort.
- Programme: Bewertet wird die Idee/Skizze und Umsetzung des Programms.

Fragen zur Prüfung:

- Während der Prüfung werden vom Dozent keine Fragen zur Prüfung beantwortet.
- Ist etwas unklar, mache eine Annahme und notiere diese auf der Prüfung.

Erste Schritte in C

1) Welche dieser Deklarationen von *main()* sind korrekt in C?

Punkte: _ / 4

Zutreffendes ankreuzen

- ☐ Ja | ☐ Nein `void main(void);`
- ☐ Ja | ☐ Nein `int main(int argc, string argv[]);`
- ☐ Ja | ☐ Nein `int main(int argc, char *argv[]);`
- ☐ Ja | ☐ Nein `int main(char *argv[], int argc);`

2) Welche dieser Ausdrücke in C sind auf Linux Systemen allgemein wahr?

Punkte: _ / 4

Zutreffendes ankreuzen, (C = C99, mit #includes):

- ☐ Ja | ☐ Nein `sizeof(unsigned int) == sizeof(int)`
- ☐ Ja | ☐ Nein `sizeof(size_t) == sizeof(long)`
- ☐ Ja | ☐ Nein `sizeof(char) < sizeof(int)`
- ☐ Ja | ☐ Nein `sizeof(int) == INT_MAX`

Funktionen in C

3) Gegeben den folgenden Code, welche Aufrufe von *eval()* sind erlaubt?

Punkte: _ / 4

```
int dec(int i) { return i - 1; }
int add(int a, int b) { return a + b; }
int eval(int a, int b, int (*op)(int, int)) { return op(a, b); }
```

Zutreffendes ankreuzen:

- ☐ Ja | ☐ Nein `eval(dec(3), 3, add);`
- ☐ Ja | ☐ Nein `eval(add(3, 3), 3, dec);`
- ☐ Ja | ☐ Nein `eval(add(3, 3), dec, add);`
- ☐ Ja | ☐ Nein `eval(eval(3, 3, add), 3, add);`

4) Schreibe ein Programm *vowels*, das die Vokale (a, e, i, o, u) zählt, welche im jeweiligen Wort vorkommen. Worte können Buchstaben (a-z) enthalten und werden per Command Line übergeben. Die Ausgabe soll pro Wort und Vokal erfolgen, wie hier im Beispiel. Punkte: __/12

```
$ ./vowels igel hase uhu  
a:0, e:1, i:1, o:0, u:0  
a:1, e:1, i:0, o:0, u:0  
a:0, e:0, i:0, o:0, u:2
```

Verwende die folgenden Calls (soweit sinnvoll), ohne *#includes* und Fehlerbehandlung:

```
int printf(const char *format, ...); // format string %s, char %c, int %d  
size_t strlen(const char *s); // calculate the length of a string
```

Idee (kurz) und C Source Code hier, oder auf Zusatzblatt mit Namen und Frage-Nr.:

File In-/Output

5) Schreibe ein Programm *append*, das am Ende einer bestehenden Datei *file* einen String *text* anhängt. Beides wird per Command Line übergeben, wie hier im Beispiel. Punkte: _ / 6

```
$ touch my.txt # empty file
$ ./append my.txt "text"
$ ./append my.txt "more"
$ cat my.txt
textmore$
```

Verwende die folgenden Calls (soweit sinnvoll), ohne *#includes* und Fehlerbehandlung:

```
size_t strlen(const char *s); // length of a string, without '\0' char
```

```
int open(const char *pathname, int flags); // Opens the file specified by
pathname. Returns the file descriptor. Flags include O_WRONLY, O_APPEND.
```

```
ssize_t write(int fd, const void *buf, size_t n); // Writes up to n bytes
from buf to the file referred to by fd. Returns nr. of bytes written ≤ n.
```

Idee (kurz) und C Source Code hier, oder auf Zusatzblatt mit Namen und Frage-Nr.:

|

|

|

|

(Aufgabe 6 auf der nächsten Seite)

6) Schreibe ein Programm *patch*, das ein existierendes File *orig* ab der Position *pos* mit Bytes eines zweiten Files *delta* beliebiger Länge überschreibt, wie hier gezeigt. Punkte: _ / 12

```
$ echo -n "hello" > orig
$ echo -n "ipp" > delta
$ ./patch orig delta 1
$ cat orig
hippo
```

Verwende die folgenden Calls (soweit sinnvoll), ohne *#includes* und Fehlerbehandlung:

```
int atoi(const char *nptr); // convert a string to an integer

off_t lseek(int fd, off_t offset, int from); // Position read/write file
offset; from = SEEK_SET, SEEK_CUR or SEEK_END; returns new offset from 0.

int open(const char *pathname, int flags); // Opens the file specified by
pathname. Returns the file descriptor. Flags include O_RDONLY, O_WRONLY.

ssize_t read(int fd, void *buf, size_t n); // Attempts to read up to n
bytes from file descriptor fd into buf. Returns number of bytes read ≤ n.

ssize_t write(int fd, const void *buf, size_t n); // Writes up to n bytes
from buf to the file referred to by fd. Returns nr. of bytes written ≤ n.
```

Idee (kurz) und C Source Code hier, oder auf Zusatzblatt mit Namen und Frage-Nr.:

Prozesse und Signale

7) Welche der folgenden Aussagen zum Stack sind im Allgemeinen korrekt? Punkte: _ / 4

Zutreffendes ankreuzen:

- ☐ Ja | ☐ Nein Der Stack hat den gesamten Speicher des Prozesses für sich allein.
- ☐ Ja | ☐ Nein Auf dem Stack allozierte Variablen überdauern Funktionsaufrufe.
- ☐ Ja | ☐ Nein Argumente und lokale Variablen werden auf dem Stack alloziert.
- ☐ Ja | ☐ Nein Rekursive Funktionsaufrufe bekommen je ein neues Stack-Frame.

8) Welche dieser Aussagen zu virtuellem Speicher sind im Allgemeinen korrekt? Punkte: _ / 4

Zutreffendes ankreuzen:

- ☐ Ja | ☐ Nein Prozess, bzw. laufendes Programm hat die CPU für sich allein.
- ☐ Ja | ☐ Nein Man muss das physische Speicherlayout nicht genau kennen.
- ☐ Ja | ☐ Nein Programm-Text kann zwischen Prozessen geshared werden.
- ☐ Ja | ☐ Nein Alle virtuellen Adressen zeigen auf gültigen phys. Speicher.

|

|

|

|

|

|

|

|

(Aufgabe 9 auf der nächsten Seite)

9) Schreibe ein Programm *nalarm*, das *n* Alarme von je 1 Sekunde abwartet und *i/n* wie unten gezeigt ausgibt. Nutze die *alarm()* Funktion, die ein SIGALRM Signal auslöst. Punkte: __ / 10

```
$ ./nalarm 3
1/3
2/3
3/3
```

Verwende die folgenden Calls (soweit sinnvoll), ohne *#includes* und Fehlerbehandlung:

```
int alarm(int sec); // arranges for a SIGALRM signal to be delivered to
the calling process in sec seconds; returns remaining sec of prev. alarm
```

```
int pause(void); // sleep until a signal causes invocation of a handler
```

```
typedef void (*sighandler_t)(int); e.g. SIGINT = 2, ^C; SIGTSTP = 20, ^Z
sighandler_t signal(int sig, sighandler_t handler); // set SIG_IGN,
SIG_DFL, or a programmer-defined function to handle the signal sig
```

```
int printf(const char *format, ...); // format string %s, char %c, int %d
```

Idee (kurz) und C Source Code hier, oder auf Zusatzblatt mit Namen und Frage-Nr.:

Prozess Lebenszyklus

10) Gegeben den folgenden Code, wie sieht der Prozess-Baum am Schluss aus? Punkte: _ / 4

```
#include ... // ignore

int main() {
    while (fork()) { break; }
    printf("%d\n", getpid());
    // <- Baum?
}
```

Zutreffende Antwort ankreuzen:

☐ Baum A	☐ Baum B	☐ Baum C	☐ Baum D
<pre> * \ \ \ * * \ \ 1 3 2 4 </pre>	<pre> * \ * \ \ \ 1 3 2 </pre>	<pre> * \ 1 2 </pre>	<pre> * \ 2 : * \ 1 n </pre>

|

|

|

|

|

|

|

|

|

|

(Aufgabe 11 auf der nächsten Seite)

11) Schreibe ein Programm *nexec*, das ein per Command Line gegebenes Programm in *n* parallelen Child Prozessen ausführt, auf deren Ende wartet, und "done." ausgibt. P.kte: _ / 12

```
$ ./nexec 3 hello
Hello, World!
Hello, World!
Hello, World!
done.
```

Verwende die folgenden Calls (soweit sinnvoll), ohne *#includes* und Fehlerbehandlung:

```
int atoi(const char *nptr); // convert a string to an integer
```

```
int execvp(char *file, char *argv[]); // executes the program referred to  
by pathname; returns only if an error has occurred; argv can be NULL.
```

```
void exit(int status); // cause normal process termination
```

```
pid_t fork(void); // create a child process, returns 0 in child process
```

```
int printf(const char *format, ...); // format string %s, char %c, int %d
```

```
pid_t wait(int *wstatus); // wait for child process to terminate; returns  
the process ID of the terminated child or -1 if no child left to wait for
```

Idee (kurz) und C Source Code hier, oder auf Zusatzblatt mit Namen und Frage-Nr.:

Threads und Synchronisation

12) Schreibe ein Programm *bubbles*, das mit Threads jede Sekunde eine Seifenblase simuliert, die losfliegt und zufällig nach 1 bis 9 Sekunden zerplatzt, wie im Beispiel unten. Punkte: _ / 14

```
$ ./bubbles
0x16f477000: fly
0x16f58f000: fly
0x16f477000: pop
...
```

Verwende die folgenden Calls (soweit sinnvoll), ohne `#includes` und Fehlerbehandlung:

```
int printf(const char *format, ...); // format string %s, pthread_t %p, ...
long random(void); // returns a random number, use value with modulo %
unsigned int sleep(unsigned int sec); // sleep for a number of seconds
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
    void *(*start) (void *), void *arg); // starts a thread; attr = NULL
int pthread_detach(pthread_t thread); // detach a thread
pthread_t pthread_self(void); // thread ID of calling thread, opaque type
```

Idee (kurz) und C Source Code hier, oder auf Zusatzblatt mit Namen und Frage-Nr.:

Zusatzblatt zu Aufgabe Nr. ____ von (Name) _____